ASSESSMENT: CO1-AT1

Course: Object-Oriented Analysis & Design (OOAD - CSA1111)

Student Name: Kalluri Gayathri
Register Number: 192372132
Submission: Assignment Assessment 1

Question 1: Library Management System — Core OO Concepts & Relationships

(a) Fundamental Object-Oriented Concepts in LMS Context

In designing a software system for a **Library Management System (LMS)** where users search, borrow, and return books:

- **Class:** A template or blueprint describing attributes and behaviors. E.g., `Book` (with attributes: *ISBN, title, author, isAvailable*) and `User` (with *userId, name*).
- **Object:** A concrete runtime instance of a class. E.g., `book1 = new Book("978-0132350884", "Clean Code", "Robert Martin")` represents a specific physical book copy.
- **Encapsulation:** Wrapping data and methods into a single unit while restricting direct state access via private fields and public accessors. E.g., marking `isAvailable` as private and toggling it exclusively through `borrowBook()` or `returnBook()` methods to prevent invalid states.
- **Inheritance:** Mechanism where a specialized subclass inherits state and behaviors from a general superclass. E.g., `StudentMember` and `FacultyMember` inherit basic attributes from the base class `User`, promoting code reuse.
- **Polymorphism:** Ability of different classes to respond to the same message with distinct behaviors. E.g., an abstract method `calculateFine()` executes one formula for `StudentMember` (e.g., $1/day) and a different formula for `FacultyMember` (e.g., grace period with reduced rate).

(b) Object Relationships in LMS

OOAD establishes semantic associations between interacting domain entities:

- **Association:** A generic bidirectional or unidirectional relationship representing a structural link. E.g., a `User` is associated with a `Book` when performing a transaction via `borrowBook()`.
- **Aggregation (Weak "has-a"):** A relationship where a whole contains parts, but the parts have an independent lifecycle. E.g., a `Library` aggregates `Book` records. If a library branch closes, the books still exist and can be cataloged elsewhere.
- **Composition (Strong "has-a"):** A strict whole-part relationship where parts depend directly on the container's lifecycle. E.g., a `Book` composed of multiple `Page` objects, or a `BorrowRecord` composed of individual `TransactionDetail` items. If the record is deleted, its constituent parts cease to exist.

(c) Modular & Reusable Architecture

Applying these OO concepts partitions the LMS into self-contained, low-coupled components. Modularity enables developers to update fine calculation algorithms or catalog search filters without destabilizing checkout workflows, thereby improving maintainability, flexibility, and scalability.

Question 2: Student Information System — Principles of OO Design

(a) Key Object-Oriented Design Principles

- **Abstraction:** Exposes essential features while hiding low-level implementation complexities. In a Student Information System, a client interacting with `EnrollmentService.enroll(studentId, courseId)` does not need to know the underlying SQL queries or validation locks.
- **Encapsulation:** Protects internal state integrity. A student's CGPA or attendance percentage cannot be directly overwritten from outside; it is recomputed via verified internal grade updates.
- **Modularity:** Decomposes a monolithic academic system into discrete, cohesive packages (e.g., *Admissions*, *Examination*, *Fee Billing*, *Hostel Portal*) with well-defined interfaces.

(b) Single Responsibility (SRP) & Open/Closed (OCP) Principles

These two fundamental SOLID design principles ensure long-term architectural stability:

- **Single Responsibility Principle (SRP):** A class should have one, and only one, reason to change. In the Student Information System:
  - *Violation:* A `Student` class that holds student profiles, generates PDF report cards, and executes database save routines.
  - *Adherence:* Decompose into `Student` (domain entity), `StudentRepository` (persistence logic), and `GradeReportGenerator` (view/formatting logic).
- **Open/Closed Principle (OCP):** Software artifacts should be open for extension but closed for modification. If a new grading policy (e.g., Relative Grading) is introduced, the system should allow extending a base `GradingStrategy` interface rather than rewriting existing tested evaluation classes.

(c) Impact on Software Quality

Adhering to SRP, OCP, and abstraction dramatically reduces system coupling, eliminates regression bugs during updates, facilitates unit testing, and improves reusability across university departments.

Question 3: Online Shopping System — Software Development Life Cycle (SDLC)

(a) Phases of SDLC for an E-Commerce Platform

- **1. Requirement Analysis:** Gathering business requirements from stakeholders (e.g., catalog search, cart checkout, multi-vendor support, payment gateway integrations). Formulated into an SRS document.
- **2. System Design:** Architecting high-level architectural schemas (microservices, cloud database models, API contracts) and low-level UML diagrams (Class, Sequence, ER diagrams).
- **3. Implementation (Coding):** Translating design specifications into working software using designated tech stacks (e.g., Java/Spring Boot, React, PostgreSQL) following coding standards.
- **4. Testing:** Executing comprehensive quality assurance verification—Unit, Integration, System, Performance (load spikes during sales events), and Security (PCI-DSS compliance) testing.
- **5. Deployment:** Releasing the verified build to staging and production server environments using CI/CD pipelines, container orchestration (Docker/Kubernetes), and CDN networks.
- **6. Maintenance:** Post-release monitoring, patching security vulnerabilities, bug fixes, and continuous feature updates based on customer analytics.

(b) Structured Development Contribution

Each phase produces formal artifacts (SRS, Design Specifications, Test Suites) that serve as explicit inputs for the next stage. This structured workflow establishes traceability, minimizes requirement ambiguity, and prevents costly late-stage architectural failures.

(c) Importance of SDLC in Delivering Reliable Software

By enforcing verification and validation at each milestone, SDLC minimizes financial risks, ensures data security for transactions, and guarantees platform scalability under high traffic volume.

Question 4: SDLC Models & Project Selection

(a) Core SDLC Models

- **Waterfall Model:** Linear-sequential approach where each phase finishes before the next begins; ideal when requirements are strictly fixed from day one.
- **Iterative Model:** Develops an initial basic version and iteratively enhances features over successive cycles until complete.
- **Spiral Model:** Risk-driven model combining iterative refinement with systematic risk assessment checkpoints at every quadrant.
- **Agile Model:** Iterative and incremental framework centered around sprints, collaboration, customer feedback, and rapid adaptability to change.

(b) Comparison Between Waterfall and Agile Models

Dimension	Waterfall Model	Agile Model
Process Structure	Sequential, rigid linear stages.	Iterative sprints (1–4 weeks).
Requirement Changes	Extremely difficult and expensive to alter mid-way.	Embraces change throughout the lifecycle.
Customer Involvement	Mainly during initial signoff and final delivery.	Continuous collaboration at every sprint review.
Working Software	Delivered only late in the lifecycle.	Produced incrementally at the end of each sprint.
Risk Management	High late-stage risk of failure.	Low risk due to frequent feedback loops.

(c) Model Recommendation for Dynamic Requirements

For systems with dynamic, evolving requirements, the **Agile Model (Scrum/Kanban)** is strongly recommended. Agile enables cross-functional teams to pivot product features without substantial rework costs.

Question 5: ATM System — Unified Modeling Language (UML) in OOAD

(a) Importance of UML in OOAD

UML provides a standardized visual modeling grammar that eliminates verbal ambiguity. It allows system architects, developers, and domain experts to visualize, specify, construct, and document the structural blueprints and runtime behaviors of an ATM system.

(b) Common UML Diagrams in an ATM System

- **Use Case Diagram:** Captures functional requirements from actor perspectives. *Actors:* Customer, Bank Admin, Maintenance Tech. *Use Cases:* `Withdraw Cash`, `Check Balance`, `Deposit Funds`, `Re-supply Cash Drawer`.
- **Class Diagram:** Illustrates static structural relationships among domain classes such as `Bank`, `Account`, `ATM`, `CardReader`, `CashDispenser`, and `Transaction`.
- **Sequence Diagram:** Models dynamic interaction over time. Demonstrates the message sequence: Customer inserts card → ATM prompts PIN → Bank verifies credentials → Cash Dispenser ejects bills.
- **Activity Diagram:** Illustrates operational workflows and conditional branches (e.g., checking if withdrawal amount ≤ daily limit and account balance ≥ request).

(c) How UML Supports Object-Oriented Methodologies

UML maps cleanly to OO concepts (classes, associations, polymorphism), bridging the gap between user stories and code-level implementation, simplifying team communication and maintenance.

Question 6: Hospital Management — Structured vs. Object-Oriented Design

(a) Structured Design Approach

A top-down functional decomposition strategy dividing a hospital system into distinct procedural modules (e.g., *Billing Module*, *Patient Module*, *Pharmacy Module*) orchestrated by centralized data flows (DFDs).

(b) Comparison: Structured vs. Object-Oriented Approach

Feature	Structured Approach	Object-Oriented Approach (OOAD)
Core Focus	Process and function-oriented (actions).	Data and object-oriented (entities + behaviors).
System Decomposition	Functional decomposition via DFDs.	Decomposition into collaborating objects/classes.
Data Handling	Shared global/external data flows.	Encapsulated private data within objects.
Reusability & Scale	Low reusability; difficult to scale safely.	High reusability via inheritance and modularity.

(c) Modular Design for Efficiency & Maintenance

Modular architectures enforce separation of concerns. In a hospital system, modifying the prescription algorithm or billing taxes is isolated to specific classes, ensuring rapid debugging, simple upgrades, and seamless third-party EHR integration.